Encapsulamiento en Taller ICNO

Proyecto Final de Programación Orientada a Objetos · Autor: Isaac Navarro · Todo el código de las clases está en **inc/taller.php**

Qué es. Encapsular es guardar los datos de un objeto bajo llave: los atributos se declaran `private` y solo se leen con un **getter** (`getCostoBase()`) o se cambian con un **setter** (`setCostoBase()`). Si una página intenta `$servicio->costoBase = -5;` , PHP lo rechaza. Así el objeto siempre controla su propio estado y, si hay que validar un dato, se hace en un solo lugar.

1 Las tres reglas que cumplen las 11 clases

Regla	En el código
a) Atributos `private`	Las 11 clases declaran `private $x;` . No existe ningún atributo `public` .
b) Acceso por getters y setters	Cada atributo tiene su `getX()` ; los que pueden cambiar tienen también `setX()` .
c) `$this` dentro de la clase	Constructores, getters y setters escriben `$this->x` : "el dato de *este* objeto".

2 Ejemplo real: clase Servicio (líneas 28 a 54)

```php
class Servicio {
    private $id;  private $descripcion;  private $costoBase;   // Bajo llave.
    public function __construct($id, $descripcion, $costoBase) {
        $this->id = $id;                          // $this = este objeto.
        $this->descripcion = $descripcion;
        $this->costoBase = $costoBase;
    }
    public function getCostoBase() { return $this->costoBase; }    // Getter
    public function setCostoBase($c) { $this->costoBase = $c; }      // Setter
    // ... igual para id y descripcion (getId/setId, getDescripcion/setDescripcion)
}
```

3 El mismo patrón en todas las clases

Clase	Atributos privados	Getters / setters
Servicio	id, descripcion, costoBase	getter y setter de los 3
ServicioPreventivo	kilometraje	getKilometraje / setKilometraje
ServicioCorrectivo	horasManoObra	getHorasManoObra / setHorasManoObra
Vehiculo	id, placas, marca, modelo, color, anio, kilometraje	getter y setter de los 7 (líneas 118 a 131)
Auto, Camioneta, Camion, Motocicleta	ninguno propio	heredan los de Vehiculo
Usuario	id, nombre, correo, rol	solo getters, a propósito
Cotizacion	id, vehiculo, servicios, total, estado, fecha	solo getters, a propósito
Taller	bd (conexión PDO)	ninguno: nunca sale de la clase

4 Las hijas también respetan lo privado del padre

Como `$costoBase` es privado en `Servicio` , ni sus hijas lo ven directo. `ServicioPreventivo` calcula con el getter del padre, y en su constructor lo del padre lo guarda el padre con `parent::__construct()` :

```php
#[\Override] public function calcularCosto() { return $this->getCostoBase() * 0.9; }  // linea 66
```

5 Decisiones de diseño: no todo tiene setter

- **Usuario** no tiene setters: nadie puede "ascenderse" con `$u->setRol('admin')` . El rol solo lo pone la base de datos.
- **Cotizacion** es un comprobante: total y servicios no se editan. Para marcarla revisada se usa `Taller->marcarRevisada()` .
- **Taller** guarda la conexión `$bd` privada y **sin getter**: ninguna página puede escribir SQL por su cuenta.
- Los métodos `*DesdeFila()` son `private` : convertir una fila SQL en objeto es asunto interno de Taller.

6 Cómo trabajan las páginas: solo métodos públicos

```php
// servicios.php 157-165: la tabla del CRUD se dibuja con getters.
<td><?php echo $s->getId(); ?></td>
<td><?php echo htmlspecialchars($s->getDescripcion()); ?></td>
<td>$<?php echo number_format($s->getCostoBase(), 2); ?></td>

// cotizar.php 34: el subtotal sale de dos metodos, sin tocar atributos.
$sub = $s->calcularCosto() * $vehiculo->getFactor();

// mis_cotizaciones.php 24: la cotizacion entrega su vehiculo como objeto.
$v = $c->getVehiculo();  echo $v->getMarca() . ' ' . $v->getModelo();
```

7 Flujo de un dato, de la base al navegador

#	Paso	Quién y con qué
1	La página pide el catálogo	`$taller->getServicios()`
2	Taller consulta MySQL	`$this->bd` privado, `prepare()` con `?`
3	Cada fila se vuelve objeto	`servicioDesdeFila()` hace `new` y el constructor llena los privados con `$this`
4	La página recibe objetos	Solo usa `getDescripcion()` , `calcularCosto()` ...
5	El admin edita	`$taller->modificarServicio($id,$d,$c)` hace el UPDATE

8 Encapsulamiento a nivel de sistema

- **Contraseñas:** `password_hash()` y `password_verify()` . Usuario ni siquiera tiene atributo de contraseña.
- **Credenciales:** en `inc/config.php` , fuera del repositorio y bloqueado por Apache (403).
- **Sesión:** solo guarda el id; `exigirRol()` pregunta al objeto con `esAdmin()` .
- **Salida HTML:** todo texto del usuario pasa por `htmlspecialchars()` .

9 Lo que se gana

Sin encapsulamiento	Con encapsulamiento
$s->costoBase = "hola";	PHP lo impide; solo `setCostoBase()` cambia el dato.
SQL repartido en cada página	Solo Taller conoce las tablas.
Un cliente cambia su rol	Usuario no tiene setRol().
Validar en 10 lugares	Validar una sola vez, en el setter.

11 clases

0 atributos públicos

100% con getters y $this

Resumen para la revisión: todos los atributos son privados y se usan con getters y setters que escriben `$this` ; los setters se omiten a propósito donde el dato no debe cambiar (Usuario, Cotizacion); la conexión a la base de datos vive privada y sin getter dentro de Taller, que es la única clase que escribe SQL. El resto del sistema trabaja solo con objetos y sus métodos públicos.